



# Save Custom Game

Documentation

---

**Creator:** Lucas Gomes Cecchini

**Pseudonym:** AGAMENOM

## Overview

This document will help you use **Assets Save Custom Game** for **Unity**.

With it you can save your game in several Slots.

It will take a **screenshot** of the game and there are three ways to save in **GameData(.json)**, **LocalLow(.json)** and **PlayerPrefs**.

It also has Auto Save features based on **Time** or **Events**.

## Instructions

You can get more information from the Playlist on YouTube:

<https://youtube.com/playlist?list=PL5hnfx09yM4JDFFdX1vfpNCLLdLq4ixZA&si=3w1IN0WrpV8IFMrS>

# Script Explanations

## Exception Utility

### General Purpose

This utility belongs to a system called **SaveCustomGame**.

Its main purpose is **to detect exactly where in the code a certain method was called from** — specifically, which file and which line.

It is used mostly for **debugging and logging**, so developers can trace back the origin of an event or an error inside the save system.

When another script calls this utility, it looks at the current call stack (the list of functions that were executed to reach that point), then filters out any internal utility frames (like `SaveDataUtility` or itself) and returns the first *external* one — that is, the one that really caused the event.

The returned information looks like this:

```
(at Assets/Scripts/Game/Player/PlayerSave.cs:128)
```

This shows the relative path inside the Unity project and the exact line where the original call happened.

### Namespace and Class

#### Namespace:

All of this is organized under the name `SaveCustomGame`.

It helps group related code dealing with save systems.

#### Class:

The utility is called `ExceptionUtility`.

It is **static**, meaning you don't need to create an object to use it.

You simply call its method directly, like `ExceptionUtility.GetCallingMethodInfo()`.

### Public Method

#### **GetCallingMethodInfo**

This is the **only public method** in the class, and the heart of the utility.

Its job is to analyze the current program's call stack and identify which file and line number triggered the chain of calls.

Let's walk through what happens inside it step by step.

### Step-by-Step Process Inside the Method

#### 1. Create a stack trace.

A stack trace is a record of all methods that have been called up to the current moment.

It's like a breadcrumb trail that shows how the program reached this point.

It also includes file names and line numbers.

Variable created:

- `stackTrace`: holds the full call history.

#### 2. Retrieve stack frames.

Each "frame" in a stack trace represents one method call.

The script extracts them from the stack trace into a list-like structure.

Variable:

- `frames`: contains all the method calls (frames) in order.

#### 3. Check if frames exist.

If for some reason there are no frames, the method will skip everything and return an empty result.

#### 4. Create a flag to track SaveDataUtility calls.

Variable:

- foundSaveDataUtility: starts as false.  
It is used later to know if the script passed through a SaveDataUtility call, which helps decide when to stop scanning the stack.

#### 5. Loop through each frame.

The method now inspects each frame in the call history, one by one.

For every frame:

- It gets the **method** associated with that frame.
- Then it checks what **class** (or type) the method belongs to.

#### 6. Filter out irrelevant internal calls.

If the class name of the frame belongs to either:

- the SaveDataUtility class, or
- the ExceptionUtility class itself,  
the frame is ignored because it's internal housekeeping.

#### 7. Find the first "external" frame.

When a frame belongs to neither of the two internal utilities, it's probably the **real code** that triggered the process (like your own script in the game).

That's the one we want.

From that frame:

- The script extracts the full **file name** (the absolute system path).
- It also extracts the **line number** inside that file.

#### 8. Validate the data.

If the file name is valid and the line number is positive (greater than zero), the script proceeds.

#### 9. Simplify the file path.

Since the full path could include the user's entire directory (like C:/Projects/MyGame/Assets/...), the script shortens it to a relative Unity path starting at the Assets folder.

It does this using a regular expression that replaces everything before "Assets".

Example:

```
C:/Projects/MyGame/Assets/Scripts/Player.cs  
becomes  
Assets/Scripts/Player.cs
```

#### 10. Format and return the result.

It creates a text line that looks like:

```
(at Assets/Scripts/Player.cs:74)
```

This is then returned by the method.

#### 11. Tracking SaveDataUtility and Exit Logic.

- If the current frame belongs to SaveDataUtility, the flag foundSaveDataUtility is set to true.
- If later the script finds that it is back inside ExceptionUtility *after* already seeing a SaveDataUtility call, it stops scanning — meaning it doesn't go deeper because it has already returned from that context.

#### 12. Fallback.

If no suitable frame was found during the whole loop, the method returns an empty string, meaning no location info was found.

## Variables Summary

| Variable             | Purpose                                                                      |
|----------------------|------------------------------------------------------------------------------|
| stackTrace           | Holds the current execution call history with file and line data.            |
| frames               | A list of method calls extracted from the stack trace.                       |
| foundSaveDataUtility | A flag indicating whether a SaveDataUtility method was seen during the scan. |
| method               | The current method being analyzed in the loop.                               |
| declaringType        | The class (type) where that method is defined.                               |
| typeName             | The full name of that class, used for filtering.                             |
| fileName             | The path to the source code file where the method resides.                   |
| lineNumber           | The line number inside that file.                                            |
| filePath             | The cleaned-up Unity-style relative path starting at “Assets”.               |

## Typical Usage

In practice, other parts of the SaveCustomGame system — for example, the file handling or error reporting modules — would call this utility whenever they need to log detailed context about where something happened.

For example, inside another script:

```
Debug.Log("Error saving data " + ExceptionUtility.GetCallingMethodInfo());
```

That would produce a log message like:

```
Error saving data (at Assets/Scripts/GameManager.cs:212)
```

This gives instant traceability in the Unity console, showing exactly where in the user’s code the save operation or exception was triggered.

## Summary

- It’s a **debugging helper** that returns the source file and line number of the **real caller**, skipping internal utilities.
- It works by examining the **stack trace** of the running program.
- It cleans up the result for readability inside Unity’s console.
- It’s **read-only** and **stateless**, meaning it doesn’t alter or store data — it just analyzes the call history and returns text.

## Save Data Utility

### General Purpose

This script acts as the **core data manager** for the *SaveCustomGame* system.

Its job is to **store, retrieve, and manage custom saved data** in a unified way.

It handles several different data types (numbers, text, booleans, and vectors) and provides extra tools for screenshots, textures, auto-save behavior, and linking scene objects related to saving.

In simpler terms:

It’s the “control center” that knows **where** and **how** game data is stored, how to **find** it again, and how to **update** it safely.

### Internal Structure Overview

Everything is grouped under a namespace called **SaveCustomGame**.

Inside it, there’s a **static utility class** named **SaveDataUtility**.

“Static” here means the class does not require being instantiated.

It behaves like a universal tool that other scripts can call directly, such as:

```
SaveDataUtility.SetInt("Player", "Health", 100)
```

or

```
float hp = SaveDataUtility.GetFloat("Player", "Health")
```

It also connects with other parts of the save system:

- SaveCustomObject — the main asset that stores all saved data.
- SaveCustomItem, SaveCustomFloat, SaveCustomVector, etc. — sub-containers used for each data type.
- SaveCustomInScene and AutoSaveCustom — components in the Unity scene that handle saving behavior.
- ExceptionUtility — used for clean and consistent debug messages that show where errors come from.



## Main Variable

### saveCustomObject

A private variable that holds a reference to the loaded *SaveCustomObject* asset.

This object contains all saved game data, stored in multiple lists depending on data type.

This variable is shared internally by all methods, ensuring every save and load operation uses the same source.



## Section 1: “Get Save Object”

### GetSaveCustomObject

This method ensures the *SaveCustomObject* asset is available before any operation.

#### What it does:

1. Checks if the internal variable saveCustomObject already holds a reference.
2. If not, it tries to load the asset from Unity’s **Resources** folder.
3. If it still fails to find it, logs an error showing the missing data issue (with the file and line where it happened).
4. Finally, returns the loaded object (or null if not found).

#### Why it matters:

All other methods depend on this one.

It guarantees that every save or load operation is referring to the correct central data container.



## Section 2: “Get Value Methods”

These are public methods used to **retrieve saved values** of different data types.

Each one uses a shared internal helper called FindValue to locate the requested entry inside *SaveCustomObject*.

#### Methods included:

- **GetVector** → returns a Vector4 value (used for vectors and rotations).
- **GetFloat** → returns a float number.
- **GetInt** → returns an integer number.
- **GetString** → returns text.
- **GetBool** → returns true or false.

#### How they work conceptually:

Each method:

1. Receives an **itemTag** (like “PlayerStats”) and a **specific key** (like “Health”).
2. Searches for a matching item inside the main save data structure.
3. Looks through the corresponding list (float list, string list, etc.).
4. If found, returns the stored value.
5. If not found, it throws an exception that clearly states which tag or item was missing.

These functions make reading saved data consistent and type-safe.

### **Section 3: “Set Value Methods”**

These are the mirror versions of the Get methods.

They allow you to **save or update values** inside the main object.

#### **Methods included:**

- **SetVector** — handles Vector2, Vector3, Vector4, or Quaternion values by converting them all into Vector4 format before saving.
- **SetVector4** — directly saves a Vector4 value.
- **SetFloat, SetInt, SetString, SetBool** — save their respective data types.

#### **How they work conceptually:**

Each one:

1. Receives an **itemTag**, a **dataTag**, and a **new value**.
2. Ensures the item exists in *SaveCustomObject*; if not, creates it automatically.
3. Ensures a list for that data type exists inside the item; if not, also creates it.
4. Searches the list for an entry matching the given dataTag:
  - If found → updates the value.
  - If not → adds a new entry to the list.

All Set methods rely on an internal generic helper method called *SetValue* that handles these steps automatically for any type.

### **Section 4: “Internal Helper Methods”**

These are private methods used internally by Get/Set operations.

They handle searching, error detection, and safe creation of data structures.

#### **FindItem**

Searches *SaveCustomObject* for an item that matches the given tag.

If not found, throws a detailed error message.

This ensures the developer knows exactly which data key is missing.

#### **FindValue**

Used by all the Get methods.

It finds and returns the actual saved value (float, int, string, etc.) from within a specific item.

It uses flexible “delegates” (functions passed as parameters) to adapt to any data type.

#### **SetValue**

Used by all the Set methods.

It either updates an existing entry or creates a new one if it doesn't exist.

It ensures the save structure remains valid (no null lists or missing items).

Together, these helpers make the entire save system generic, consistent, and easy to expand.

### **Section 5: “Screenshot & Texture Methods”**

This section handles the **visual capture and conversion** of game screenshots, which are stored as image data inside *SaveCustomObject*.

#### **CaptureScreenshot**

Captures a screenshot from a given camera and stores it inside the main save object.

#### **Process overview:**

1. Validates the camera.
2. Calculates screen size and scales it to avoid large textures.
3. Creates a temporary render target to capture the camera output.
4. Copies the image data into a new texture.
5. Converts that texture to PNG format and stores it as bytes in the save object.
6. Cleans up memory afterward.

This allows you to store an image representation of the game state (for example, a save-game thumbnail).

#### **RenderScreenshot**

Takes the saved PNG byte array and reconstructs it into a Texture2D image in memory.

If the byte array is invalid or missing, it logs an error.

## TextureToSprite

Converts a Texture2D image into a Unity Sprite object so it can be displayed in the UI. If the texture is missing, it logs an error instead of crashing.

## ✖ Section 6: “Scene Component & Auto-Save”

This part connects the save system logic with Unity scene objects.

### GetComponentSaveCustomInScene

Searches for a GameObject named “[Save Custom Object]” in the current scene.

If found, it tries to get the component SaveCustomInScene from it.

If the object or component does not exist, it logs a detailed error.

This is useful when scripts need to interact with the scene-level save handler.

### SetAutoSave

Enables or disables automatic saving behavior by setting a flag (autosaveEnabled) inside the *SaveCustomObject* asset.

### SaveEvent

Looks for the “[Save Custom Object]” in the scene again.

If found and it has the AutoSaveCustom component, it tells that component to run its

**SaveAutoGame** method, triggering an immediate save.

If the object or component is missing, it logs an appropriate error message.

## 📖 Summary of All Variables and Their Roles

| Variable                                            | Purpose                                                                                                |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| saveCustomObject                                    | Holds the main data asset used for storing all save information.                                       |
| itemTag                                             | A string identifier grouping related values (like “PlayerStats”).                                      |
| dataTag                                             | A specific key identifying an individual piece of data (like “Health”).                                |
| newValue                                            | The value being saved or updated.                                                                      |
| saveObject                                          | Temporary reference to the main <i>SaveCustomObject</i> inside helper methods.                         |
| list                                                | The specific list inside an item that stores entries of a certain data type.                           |
| entry                                               | A single element (e.g., a float, int, or vector) inside one of those lists.                            |
| getList / setList / getTag / setValue / createEntry | Internal function references used by generic helper methods to interact with the right data structure. |
| targetCamera                                        | The camera used to render and capture screenshots.                                                     |
| rt (RenderTexture)                                  | Temporary texture that stores the camera’s rendered frame.                                             |
| screenshot (Texture2D)                              | The final captured image before conversion to PNG.                                                     |
| aspect, width, height                               | Values used to maintain correct proportions during screenshot resizing.                                |

## 🧠 Typical Usage in a Game

Here’s how this utility would be used conceptually:

- **Saving a player’s score:**

```
SaveDataUtility.SetInt("PlayerStats", "Score", 1200)
```

- Loading that score later:

```
int score = SaveDataUtility.GetInt("PlayerStats", "Score")
```

- Capturing a thumbnail when saving:

```
SaveDataUtility.CaptureScreenshot(mainCamera)
```

- Displaying the saved screenshot in the menu:

```
var tex = SaveDataUtility.RenderScreenshot(saveObject.screenshot)
var sprite = SaveDataUtility.TextureToSprite(tex)
```

- Triggering an auto-save:

```
SaveDataUtility.SaveEvent()
```

All of this happens through one unified interface that automatically handles safety checks, creation of missing data containers, and detailed logging.

## Final Overview

In essence:

| Feature                 | Description                                                             |
|-------------------------|-------------------------------------------------------------------------|
| Central Save Management | Controls all reading and writing to the main save asset.                |
| Type-Independent Design | Uses generic methods to support all data types with minimal repetition. |
| Safe and Robust         | Checks for nulls, missing tags, and missing components.                 |
| Debug-Friendly          | Every log message shows exactly where in the code it originated.        |
| Extra Utilities         | Manages screenshots, textures, and auto-save functions.                 |

## Create Base Script

### General Purpose

This script is a **Unity Editor utility** designed to help developers quickly create **two essential scripts** used by the save system:

1. **Game Save Manager** — handles how game data is saved.
2. **Game Load Manager** — handles how game data is loaded and restored.

It does this automatically by generating the corresponding code files directly in the Unity project when the user clicks options in the **Unity Editor menu**.

This eliminates the need for manual setup, prevents mistakes, and ensures that the generated scripts always follow a consistent format.

### Structure Overview

Everything is placed under the namespace **SaveCustomGame** to keep it grouped with the other system utilities.

Inside it, there is a single class: **CreateBaseScript**.

This class is **not used at runtime** — it runs only inside the Unity Editor.

It uses Unity's **Editor menu system** and the **AssetDatabase API** to interact with project files.

### Internal Method: "Create Script"

**CreateScript (baseName, baseCode)**

This is the **core method** of the utility — the one that actually creates new script files in your Unity project.

### Purpose

To take a name and a block of source code and turn them into a new .cs file directly in the Unity project folder.

### How It Works (step by step)

1. **Determine the Target Folder**
  - It checks what the user currently has selected in the Unity Project window.
  - If nothing is selected, it defaults to the main Assets folder.
  - If a folder is selected, it uses that folder.
  - If a file is selected, it takes the folder where that file is located.
2. **Build the Final File Path**
  - Combines the selected path with the chosen script name to create a valid file path ending in “.cs”.
3. **Check for Existing File**
  - If a file with the same name already exists in that folder, it stops and logs a warning — preventing accidental overwriting.
4. **Write the File**
  - Writes the entire block of source code into the new file using normal file I/O operations.
5. **Update Unity’s Database**
  - Calls a refresh function so Unity recognizes the new script immediately without needing a manual refresh.
6. **Load and Highlight the New File**
  - Loads the new script into Unity’s asset system and automatically selects it in the Project view.
7. **Confirmation Log**
  - Prints a message in the Unity Console confirming that the script was created successfully.

### Variables Used

| Variable       | Description                                                                        |
|----------------|------------------------------------------------------------------------------------|
| selectedPath   | Stores the current folder or file path selected by the user in the Project window. |
| selectedObject | The currently selected item in the Unity Editor.                                   |
| scriptPath     | The full destination path of the new script (folder + filename).                   |
| asset          | Reference to the newly created script asset once Unity detects it.                 |

### Usage

You never call this function manually.

It is used internally by the menu commands described below.

### Menu Section

This section contains the Unity Editor menu entries that appear under:

**Assets → Create → Save Custom Game → ...**

When a developer clicks one of these options, a prewritten script is automatically created.

### “CreateSaveManager” Menu Command

#### Menu Path:

Assets → Create → Save Custom Game → Game Save Manager (Script)

### Purpose

Creates a ready-to-use script called **GameSaveManager** — the one responsible for saving player data, object states, and general game state.

### How It Works

When selected:

1. Defines a long text block containing the complete source code of the **GameSaveManager**.
2. Calls the internal **CreateScript** function, passing:
  - "GameSaveManager" as the file name.
  - The full text block as the script content.

### The Generated “GameSaveManager” Script

Here’s a conceptual summary of what the generated script does (you don’t need to know code syntax):

- It manages the saving of **player data**, **scene data**, and **object states**.
- It uses the **SaveDataUtility** (another part of the system) to store and retrieve all values.
- It organizes keys into groups:
  - *GameSave* for global flags (like “IsAllowed”).
  - *Player* for position and rotation.
  - *Object* for active/inactive states.
- It includes methods such as:
  - **ClearAllSaveData** — resets all data to defaults.
  - **SetLoadingSave / GetLoadingSave** — enable or disable load permissions.
  - **SetPlayerData / GetPlayerData** — save and restore player transform.
  - **SetObjectsData / GetObjectsData** — save and restore active states of scene objects.
- It ensures safe data handling and provides debug logs for invalid or missing data.

The generated GameSaveManager acts as the **core logic** behind saving gameplay progress.

## 2 “CreateLoadManager” Menu Command

### Menu Path:

Assets → Create → Save Custom Game → Game Load Manager (Script)

### Purpose

Creates another prebuilt script called **GameLoadManager**, which works together with the GameSaveManager.

### How It Works

When selected:

1. Defines another large block of text that represents the full content of the GameLoadManager.
2. Calls the **CreateScript** method to create the file "GameLoadManager.cs" in the chosen folder.

### The Generated “GameLoadManager” Script

Conceptually, this script:

- Is a Unity **component** (can be added to GameObjects).
- Handles **loading** previously saved data and **restoring** game states.
- Works side-by-side with the GameSaveManager.
- Automatically loads the saved game on scene start (if allowed).
- Includes safety checks for missing references or objects.
- Provides tools for:
  - Saving game state.
  - Loading game state.
  - Reloading the current scene.
  - Running restart events through UnityEvents.
- Includes an **editor-only inspector** with a “Clear Save Data” button for convenience.

This generated script forms the **in-scene controller** that manages loading and scene reinitialization.

## Summary of How the Utility Works

| Step | Action                            | Description                    |
|------|-----------------------------------|--------------------------------|
| 1    | User opens Unity’s Project window | Navigates to a folder or file. |

|   |                                 |                                                             |
|---|---------------------------------|-------------------------------------------------------------|
| 2 | User clicks menu option         | Either “Game Save Manager” or “Game Load Manager”.          |
| 3 | The CreateBaseScript class runs | Uses the CreateScript method internally.                    |
| 4 | File is written                 | The chosen script is created with complete source code.     |
| 5 | Unity refreshes                 | The Editor detects and compiles the new file automatically. |
| 6 | Script is highlighted           | The user sees it selected in the Project view.              |

### Conceptual Roles of Each Component

| Component                   | Purpose                                                         |
|-----------------------------|-----------------------------------------------------------------|
| CreateBaseScript            | The central utility that generates script files from templates. |
| CreateScript                | The worker method that writes a file into the project folder.   |
| MenuItem Attributes         | Create clickable options inside the Unity Editor.               |
| GameSaveManager (generated) | Manages all data saving operations.                             |
| GameLoadManager (generated) | Handles loading, restoring, and editor utilities.               |

### When and How You Use It

#### When:

You use this utility when setting up the Save Custom Game system for the first time or when you want to recreate the base management scripts.

#### How:

1. In Unity, open the **Assets** menu.
2. Go to **Create** → **Save Custom Game** → **Game Save Manager (Script)** or **Game Load Manager (Script)**.
3. The script file appears automatically inside the selected folder.

No coding required — both scripts are ready to use immediately.

### Summary in One Sentence

**CreateBaseScript** is a Unity Editor tool that automatically writes and installs the two main controller scripts (GameSaveManager and GameLoadManager) needed for the Save Custom Game system, saving time and ensuring consistency across projects.

## Open Save Custom Data

### General Purpose

This script adds a **new menu option** inside Unity’s toolbar.

When you click it, it **automatically opens** the *SaveCustomData* asset — which is a **ScriptableObject** that stores all the configuration used by the **Save Custom Game** system. In short:

It’s a shortcut to open the save system settings instantly, without needing to search for them manually in the Project window.

### Structure Overview

- **Namespace:** SaveCustomGame  
Keeps the class organized with other scripts of the same save system.
- **Class:** OpenSaveCustomData  
This is a **tool class**, not something you attach to GameObjects.  
It only works in the Unity Editor and is never included in the built game.

### Public Method — “OpenSettingsData”

This is the **only method** in the class and the main logic of the entire script. It is marked with a special Unity Editor tag so that it appears as a menu command under: **Window → Save Custom Game → Save Custom Settings**. When you click that menu option, Unity runs this method.

### Step-by-Step Explanation

Let's go through what happens inside the method line by line.

#### **1 Retrieve the Save Configuration Object**

**Variable:** settingsData

This variable stores the result of a call to another utility function — the **SaveDataUtility.GetSaveCustomObject()** method.

That method locates and returns the **SaveCustomData** asset used by the save system. If it finds the ScriptableObject successfully, settingsData will hold a reference to it. If not, it will be empty (null).

- **Purpose:** To access the main configuration asset that defines how saving works in the project.

#### **2 Check if the Asset Exists**

If settingsData is empty (meaning the configuration asset doesn't exist), the script:

- Shows an error message in the Unity Console:  
    *"Failed to load SaveCustomData. Ensure the ScriptableObject exists."*
- Then stops running the method.

This prevents the tool from trying to open something that doesn't exist.

#### **3 Search for an Already Open Window**

**Variable:** existingWindow

This step looks through all **currently open editor windows** in Unity.

It uses Unity's **Resources** system to get every open window and then looks for one whose **title text** matches the name of the settingsData asset.

- **Purpose:**  
    To check if the configuration file is **already open** in another tab or window, avoiding opening duplicate inspectors.

If it finds one, the reference to that window is stored in the variable existingWindow.

#### **4 Focus the Window if It's Already Open**

If existingWindow contains a valid window:

- Unity brings that window to the front and focuses it.  
    This ensures you don't accidentally open multiple tabs showing the same asset.

#### **5 Open the Settings if No Window Exists**

If no existing window is found:

- The script uses Unity's **EditorUtility** tools to **open the SaveCustomData asset** in a **Property Editor** window.

That window shows all the fields and settings stored in the SaveCustomData asset, letting you view or change them directly.

### Variables Used Inside the Method

| Variable       | Description                                                                          | Variable       |
|----------------|--------------------------------------------------------------------------------------|----------------|
| settingsData   | Holds the reference to the SaveCustomData asset retrieved from the utility.          | settingsData   |
| existingWindow | Stores a reference to any currently open Unity Editor window showing the same asset. | existingWindow |

## How It Works in Practice

When using this tool inside Unity:

1. Go to the top toolbar and click:  
**Window → Save Custom Game → Save Custom Settings**
2. The tool looks for your **SaveCustomData** asset automatically.
  - If it doesn't exist, you'll see an error in the console.
  - If it exists, Unity either:
    - Focuses the already open window (if you already have it open), or
    - Opens a new Property Editor showing that asset.

This provides a **single-click shortcut** to access your game's save configuration.

## Purpose in the Save Custom Game System

The **SaveCustomData** asset holds the **core settings** for how game data is saved, like:

- Save file paths.
- Encryption settings.
- Version identifiers.
- Other system-level options.

Normally, you would have to search for that ScriptableObject manually in your project folders. This script saves time by opening it immediately from the menu.

## Summary of Method Flow

| Step | Action                   | Description                                   |
|------|--------------------------|-----------------------------------------------|
| 1    | Retrieve asset           | Gets the SaveCustomData configuration object. |
| 2    | Validate asset           | Checks that it exists before continuing.      |
| 3    | Look for existing window | Ensures you don't open duplicates.            |
| 4    | Focus window             | Brings the current settings tab to the front. |
| 5    | Open new window          | Opens the Property Editor for SaveCustomData. |

## Summary of Usage

| Element       | Description                                              | Element       |
|---------------|----------------------------------------------------------|---------------|
| Menu Location | Window → Save Custom Game → Save Custom Settings         | Menu Location |
| Action        | Opens or focuses the SaveCustomData configuration.       | Action        |
| When to Use   | Anytime you want to adjust save system settings quickly. | When to Use   |
| Who Uses It   | Developers or designers configuring how saving behaves.  | Who Uses It   |

## Summary in One Sentence

**OpenSaveCustomData** adds a convenient Unity Editor menu option that automatically locates and opens the project's save configuration asset (*SaveCustomData*), letting developers access and edit it instantly without manually searching in the project.

## Prefab Creator

### General Purpose

This utility provides **ready-made menu options** in the Unity Editor so developers can instantly place a **Save** or **Load** UI panel into their scene.

It manages everything automatically:

- It ensures there's a proper **Canvas** and **EventSystem** in the scene.
- It finds the correct **prefab file** in your project.
- It **instantiates** that prefab correctly.

- It **selects and prepares** the new object for editing.

This saves time and ensures all Save Custom Game UI elements are always created consistently and correctly.

## Structure Overview

Everything is organized inside the namespace **SaveCustomGame**, keeping it part of the same system as the other save utilities.

The tool consists of a **single static class** called **PrefabCreator**, containing multiple **utility methods** grouped into regions.

- The **first region** deals with internal logic (Canvas creation, prefab finding, prefab setup).
- The **second and third regions** define **menu commands** for Unity's Editor to call those internal methods.

This class is **not used at runtime** — it only runs inside the **Unity Editor**.

## Region: Canvas and Prefab Creation

This is the core logic responsible for setting up and spawning UI prefabs.

### Method: "CreateUICanvas"

#### Purpose

Creates a new **UI Canvas** and **EventSystem** if the scene doesn't already have one.

A Canvas is the root object required by Unity to display any UI element (buttons, panels, etc.).

An EventSystem handles clicks, input fields, and other interactions.

#### Step-by-Step Operation

1. Creates a new object called "Canvas".
2. Adds Unity's UI components to it:
  - **Canvas** — main renderer for all UI.
  - **CanvasScaler** — adjusts scaling for different resolutions.
  - **GraphicRaycaster** — detects and handles UI input.
3. Configures it to be visible on top of everything (screen space overlay).
4. Ensures it is on the **UI layer**.
5. Creates an **EventSystem** and attaches required input modules so the UI can receive events.
6. Registers both objects with Unity's **Undo system**, allowing you to undo creation actions.
7. Returns a reference to the newly created Canvas.

#### Variables Used

| Variable      | Description                                                     |
|---------------|-----------------------------------------------------------------|
| canvasGO      | Stores the GameObject that holds the Canvas component.          |
| canvas        | The actual Canvas component used for displaying UI.             |
| eventSystemGO | Stores the EventSystem GameObject created to handle user input. |

#### Usage

Automatically called whenever a prefab is added but no Canvas exists in the scene.

You never call this method manually.

### Method: "FindPrefabByName"

#### Purpose

Searches the Unity project files for a prefab by name, specifically inside the **Save Custom Game/Prefab** folder.

#### Step-by-Step Operation

1. Scans all project assets looking for prefabs with a matching name.
2. Converts internal Unity IDs into actual file paths.
3. Checks if the prefab is located inside the correct folder and matches the given name.
4. Loads and returns that prefab so it can be instantiated.
5. If not found, prints an error message in the console.

## Variables Used

| Variable | Description                                                    |
|----------|----------------------------------------------------------------|
| guids    | A list of all prefab identifiers returned by the asset search. |
| path     | The real file path of each prefab being checked.               |

## Usage

Used internally by the next method to locate the correct prefab file before creating it in the scene.

### 3 Method: “CreateAndConfigurePrefab”

#### Purpose

Handles the **main prefab creation process** — finds the prefab, ensures a Canvas exists, instantiates the object, and configures it.

#### Step-by-Step Operation

1. Checks the current scene for an existing Canvas.
  - If none is found, it calls **CreateUICanvas** to generate one.
2. Looks for the specified prefab by calling **FindPrefabByName**.
3. If the prefab doesn't exist, it stops and logs an error.
4. Decides where the new prefab will be placed:
  - If the user has a GameObject selected in the Hierarchy, the prefab becomes its child.
  - Otherwise, it attaches the prefab under the main Canvas.
5. Instantiates (creates) the prefab inside the scene.
6. Calls another method — **FinalizePrefabSetup** — to finish the process.

## Variables Used

| Variable | Description                                                                              |
|----------|------------------------------------------------------------------------------------------|
| canvas   | Holds a reference to an existing or newly created Canvas.                                |
| prefab   | The prefab asset loaded from the project folder.                                         |
| parent   | The Transform that will hold the new prefab instance (either selected object or Canvas). |
| instance | The actual new GameObject created from the prefab.                                       |

## Usage

This is the main internal method used by all menu commands to insert UI prefabs into the scene.

### 4 Method: “FinalizePrefabSetup”

#### Purpose

Completes the prefab creation process by making it editable, selectable, and ready for immediate renaming.

#### Step-by-Step Operation

1. Registers the created object with Unity's Undo system, allowing it to be undone like any normal editor action.
2. “Unpacks” the prefab, converting it into a normal GameObject hierarchy.
  - This allows developers to freely edit it without changing the original prefab file.
3. Automatically selects the new prefab in the Hierarchy.
4. Triggers Unity's rename mode (as if you pressed **F2**) to let you rename it immediately.

## Variables Used

| Variable      | Description                                                      |
|---------------|------------------------------------------------------------------|
| fileName      | The base name of the prefab (used in logging and Undo messages). |
| newGameObject | The new instance created from the prefab.                        |

## Usage

Automatically called by **CreateAndConfigurePrefab** — developers don't call it directly.

## Region: Legacy UI Prefabs

These are **menu entries** that appear inside Unity under:

**GameObject** → **UI** → **Save Custom Game** → **Legacy**

They allow you to add the **old-style UI** prefabs for saving and loading menus (using Unity's classic UI Text system).

### Menu Items

| Menu Option        | What It Does                                    |
|--------------------|-------------------------------------------------|
| Load Menu (Legacy) | Instantiates the old-style load menu UI prefab. |
| Save Menu (Legacy) | Instantiates the old-style save menu UI prefab. |

Both call the same internal method (**CreateAndConfigurePrefab**) with different prefab names.

## Region: TMP UI Prefabs

These are similar to the Legacy options but use **TextMeshPro (TMP)**, which provides sharper and more modern text rendering.

They appear in the Unity Editor under:

**GameObject** → **UI** → **Save Custom Game**

### Menu Items

| Menu Option     | What It Does                                      |
|-----------------|---------------------------------------------------|
| Load Menu (TMP) | Creates the modern load menu with TextMeshPro UI. |
| Save Menu (TMP) | Creates the modern save menu with TextMeshPro UI. |

Each one again calls **CreateAndConfigurePrefab** with the proper prefab name (for example, "Save Menu (TMP)").

## How It Works in Practice

Here's what happens from the user's perspective:

1. In Unity's top toolbar, open:
    - **GameObject** → **UI** → **Save Custom Game** → **Load Menu (TMP)**
    - or
    - **GameObject** → **UI** → **Save Custom Game** → **Save Menu (TMP)**
  2. The tool checks if the scene has a Canvas and EventSystem.
    - If not, it creates them automatically.
  3. It finds the matching prefab in your **Save Custom Game/Prefab** folder.
  4. The prefab is instantiated into the scene, automatically selected, and ready for renaming.
- Within a few seconds, you have a fully functional Save or Load menu in your scene.

## Summary of All Methods and Variables

| Method                                      | Description                                | Type    |
|---------------------------------------------|--------------------------------------------|---------|
| CreateUICanvas                              | Creates Canvas and EventSystem if missing. | Private |
| FindPrefabByName                            | Searches project for prefab by name.       | Private |
| CreateAndConfigurePrefab                    | Finds, creates, and places prefab.         | Private |
| FinalizePrefabSetup                         | Finishes setup, selection, and rename.     | Private |
| CreateLoadMenuPrefab / SaveMenuPrefab       | Menu commands for legacy prefabs.          | Public  |
| CreateLoadMenuPrefabTMP / SaveMenuPrefabTMP | Menu commands for TMP prefabs.             | Public  |

## Usage Summary

| Element            | Description                                                       |
|--------------------|-------------------------------------------------------------------|
| Where to Find It   | Unity Editor → GameObject → UI → Save Custom Game                 |
| Purpose            | Instantly add Save/Load UI menus to your scene.                   |
| Automatic Behavior | Creates Canvas and EventSystem if missing.                        |
| Prefab Source      | Loads prefabs from "Save Custom Game/Prefab" folder.              |
| Customization      | After creation, you can immediately rename and modify the prefab. |

## Summary in One Sentence

**PrefabCreator** is a Unity Editor utility that automatically locates, creates, and configures Save Custom Game UI prefabs — ensuring that save/load menus can be added to a scene instantly, complete with Canvas and EventSystem setup.

## Auto Save Custom

This script defines a Unity component that automatically saves game data at regular time intervals while the game is running. It is designed to operate quietly in the background, managing multiple autosave slots to prevent overwriting a single save file repeatedly.

### Purpose and Overview

The component provides **automated saving** for a game that uses the “Save Custom Game” system. It ensures the game periodically stores progress in different save slots, cycling through them to maintain a rotating backup system.

It references another component, `SaveCustomInScene`, which performs the actual save operations.

You would typically attach this script to a persistent object in your scene, such as a game manager or controller. Once active, it continuously checks elapsed time and automatically triggers a save when the configured interval passes.

### Variables and Their Roles

#### Public Variables

- **saveCustomInScene**

This is a reference to a `SaveCustomInScene` component in the scene.

That component handles the actual process of saving and loading data.

If this reference is not manually assigned, the script automatically tries to locate one when it starts.

#### Private Variables

- **currentAutoSaveSlot**

An integer that keeps track of the current autosave slot number.

The system cycles through these slots (from 1 to 6), so older autosaves get replaced in a loop.

Example: first save → slot 1, next save → slot 2, and after slot 6, it goes back to slot 1.

- **timeSinceLastSave**

A float value representing how many seconds have passed since the last autosave.

It increases over time and resets every time an autosave happens.

- **saveInterval**

Defines the number of seconds between autosaves.

This value is fetched from the project's custom save settings (via `SaveCustomObject`).

- **saveKey**

A constant string used as a key in Unity's `PlayerPrefs` system.

It stores which autosave slot was used last, so the next autosave continues from there even after restarting the game.

### Methods and Their Functions

#### Start()

This method runs once when the component is initialized.

It performs setup tasks to prepare the autosave system:

1. Checks if `saveCustomInScene` is missing.  
If so, it automatically retrieves a valid reference using a utility function.
2. Disables autosaving initially to prevent it from starting unintentionally.

3. Reads the autosave interval (the time between saves) from the project's save settings.
4. Retrieves the last used autosave slot from PlayerPrefs, ensuring continuity across play sessions.

### FixedUpdate()

This method runs at a fixed time step — ideal for consistent time tracking independent of frame rate.

It handles the timing logic for when to trigger an autosave.

1. Adds the elapsed fixed time (Time.fixedDeltaTime) to timeSinceLastSave.
2. Checks whether the accumulated time has reached or exceeded the saveInterval.
3. If so:
  - Resets the timer.
  - Verifies that the system is not already using event-based saving (saveGameByEvent must be false).
  - If conditions are met, it triggers the autosave process by calling SaveAutoGame().

### SaveAutoGame()

This method performs the actual autosave process.

It is only executed when autosaving is enabled in the configuration.

Here's what it does step-by-step:

1. Checks if autosaving is allowed (autosaveEnabled must be true). If not, it exits immediately.
2. Builds a filename based on the current autosave slot, for example: "0 - 3" → representing slot 3.
3. Calls the SaveData() method from the SaveCustomInScene component to actually write the game data to disk.
4. Updates the slot counter — cycling through 1 to 6 repeatedly.
5. Saves the new slot number into PlayerPrefs so the rotation continues seamlessly in future play sessions.

### Usage

- Attach this component to a persistent object (for example, a "Game Manager").
- Assign a SaveCustomInScene reference in the inspector or let it be found automatically.
- Configure the autosave interval and other settings inside the SaveCustomObject (your project's main save configuration file).
- Enable the autosave option in that configuration.
- Once the game starts, it will automatically save at the defined intervals.

### Summary of Workflow

1. The script initializes and loads configuration data.
2. It counts time in the background during gameplay.
3. When enough time has passed, it performs a save operation using SaveCustomInScene.
4. It cycles through multiple slots, preventing a single file from being constantly overwritten.
5. It remembers the current slot between play sessions via PlayerPrefs.

## Save Custom Initialization

This script is responsible for **automatically setting up and initializing the Save Custom Game system** when the game starts. It loads configuration data, creates the runtime save manager object, attaches essential saving components, and ensures that the save system persists across scenes.

It is designed to work entirely in the background so that developers do not need to manually add these objects to every scene.

## General Purpose

The main goal of this script is to guarantee that the saving system is **ready as soon as the game begins**, even before any scene objects are interacted with. It uses Unity's runtime initialization attribute to execute automatically when the application launches. It establishes a persistent GameObject that stays alive across scene changes, ensuring consistent and centralized save functionality for the entire game session.

## Variable Explanation

### saveCustomObject

- This is a **static variable**, meaning it belongs to the class itself rather than any instance.
- It stores the main configuration data for the saving system, represented by a special asset file (SaveCustomObject).
- The asset is loaded from the game's **Resources** folder, where Unity stores global data accessible during runtime.
- Every save operation and rule (like autosave settings, file names, and behaviors) is defined within this object.

Essentially, this variable acts as the **core settings reference** used by the other save-related components.

## Methods Overview

The script has two main methods that work together to initialize and prepare the saving system:

### RunGameInitialization()

This is the **entry point** that executes automatically when the game starts — it doesn't require a manual call.

The attribute above it instructs Unity to run this method during the initial loading stage.

### Step-by-step Explanation:

1. **Console Log (Startup Confirmation):**  
Prints a message in the Unity console to confirm that the save system initialization process has begun.
2. **Load Configuration Data:**  
Calls another internal method (LoadSettingsData()) that attempts to locate and load the SaveCustomObject asset from the Resources folder.
3. **Error Handling:**  
If the system cannot find the required asset, it logs an error message and aborts the initialization process to avoid creating an incomplete save manager.
4. **Create Persistent Manager Object:**  
A new GameObject is created in the scene named "[Save Custom Object]". This object will act as the container for all runtime saving components.
5. **Attach Components:**  
Two essential components are attached to this GameObject:
  - SaveCustomInScene: The main component responsible for handling data serialization, file saving, and loading.
  - AutoSaveCustom: A complementary component that manages timed background autosaving.
6. **Link Data References:**
  - The SaveCustomInScene component receives the loaded SaveCustomObject as its configuration source.
  - The AutoSaveCustom component is linked to the SaveCustomInScene so that it can perform autosaves using the correct save logic.
7. **Persist Across Scenes:**  
The object is marked as **non-destructible**, meaning it will not be removed when a new scene loads.

This ensures that the save system remains active globally throughout the entire gameplay session.

In short, this method automatically builds the **runtime environment** that powers the saving system, guaranteeing everything is ready and connected before gameplay begins.

### LoadSettingsData()

This method's responsibility is to **retrieve the save configuration asset** from Unity's Resources folder.

#### How It Works:

1. **Loading Attempt:**  
It searches for a file named "**Save Custom Object Data**" inside the Resources directory. This file should be a ScriptableObject asset containing all the configuration data for the save system.
2. **Assignment:**  
If the file exists, it is loaded and assigned to the saveCustomObject variable.
3. **Error Handling:**  
If it cannot find the asset, the method logs an error to the console to alert the developer that the required resource is missing.

This ensures that all saving components rely on a single, consistent configuration file.

### Usage

You do not need to attach this script to any object in the scene.

Its design allows it to run **automatically** when the game starts, thanks to the runtime initialization feature.

To use it correctly:

1. Place a Save Custom Object Data asset inside a **Resources** folder in your Unity project. This file defines all parameters related to the save system.
2. Ensure your project includes the SaveCustomInScene and AutoSaveCustom components.
3. When the game launches, the system will automatically:
  - Load the configuration data,
  - Create the persistent manager object,
  - Attach and connect the necessary components.

### Summary of Functionality

| Step | Action               | Description                                              |
|------|----------------------|----------------------------------------------------------|
| 1    | Game Starts          | The script runs automatically on startup.                |
| 2    | Loads Settings       | Retrieves the configuration asset from Resources.        |
| 3    | Creates Object       | Builds a new persistent manager GameObject in the scene. |
| 4    | Attaches Components  | Adds the save and autosave scripts to the object.        |
| 5    | Links References     | Connects the components so they share the same settings. |
| 6    | Prevents Destruction | Keeps the manager alive across all scenes.               |

### In Summary

The script acts as the **entry gate** for the entire Save Custom Game system.

It automatically configures and launches the saving framework at runtime, ensuring:

- A consistent saving mechanism across all game scenes.
- Automatic access to save settings and autosave logic.
- No need for manual setup in each individual scene.

It's a **self-initializing, persistent manager** that ensures the save system is always active, properly configured, and ready to operate from the very start of the game.

## Save Custom In Scene

## General Overview

This script is part of a **custom game save system**.

It runs inside a game scene and is responsible for:

- Tracking how long the player has been in the game.
- Capturing a screenshot for the save slot.
- Recording which scenes are currently loaded.
- Saving all this information either to a file on disk or to Unity's PlayerPrefs.
- Loading the same data back when the game resumes.

The system works together with another object called **SaveCustomObject**, which stores configuration and data for saving (such as settings and item lists).

It's designed to work automatically in the background — usually attached to a persistent object in the scene.

## Variables and Their Purposes

### Public Fields (visible in the Unity Inspector)

- **saveCustomObject**  
Reference to the main data container that holds all information about the save system (like settings, items, scenes, and screenshot).
- **fileName**  
The name used for the save file. For example: "SavingData" creates "SavingData.json" when stored.
- **savePath**  
The complete location where the save file will be stored. It can be automatically filled depending on configuration.
- **sceneNames**  
A list containing the names of all currently active scenes when a save occurs. This helps restore multiple scenes during loading.
- **sceneCamera**  
A camera used to take a screenshot for the save thumbnail. If none is assigned, the script will try to find one automatically.

### Private Fields

- **gameTime**  
A formatted text version of how long the game has been running (for example "01:32:47").
- **elapsedTime**  
A numeric counter that increases as time passes. It's used to calculate the formatted game time.

## Public Methods (Main Functionalities)

### ResetSave()

Resets all save-related data:

- Sets the elapsed time back to zero.
- Clears the screenshot and scene list from the save object.
- Resets the displayed game time text to "00:00:00".

### Usage:

Called when the player starts a new game or clears a save slot.

### SaveData()

Captures and stores the current game state. It does several steps:

1. **Camera Handling:**  
If there is no camera assigned, it automatically searches for one in the scene (player camera, main camera, or any available one).
2. **Screenshot Capture:**  
Takes a screenshot from the chosen camera and stores it as a byte array in the save object.

### 3. **Data Preparation:**

Creates a serializable container called `SaveCustomFile` that gathers:

- Screenshot.
- Game time.
- Scene names.
- List of saved items.
- Elapsed time.

### 4. **Data Serialization:**

Converts this container into a JSON string so it can be saved easily.

### 5. **Saving Logic:**

Depending on configuration inside `SaveCustomObject`, it chooses one of three locations:

- **LocalLow folder:** Common for standalone builds (persistent data path).
- **PlayerPrefs:** Stores JSON in Unity's internal key-value system (no external file).
- **Project or build folder:** For local or editor testing.

### 6. **Directory Handling:**

Ensures that the target directory exists before writing.

### 7. **File Writing:**

Writes the JSON string to the chosen path.

### 8. **Cleanup:**

Clears the stored camera reference to prevent unwanted reuse later.

### **Usage:**

Called when the player manually saves the game, or during auto-save events.

### **LoadData()**

Loads previously saved information. The process goes as follows:

#### 1. **Read Source:**

If using `PlayerPrefs`, it retrieves the JSON string stored under the file name.

If saving to disk, it reads the JSON file from the save path.

#### 2. **Deserialize:**

Converts the JSON text back into a structured data container.

#### 3. **Apply Data:**

Passes the loaded information to an internal function (`LoadDataString`) that restores everything in the scene and save object.

### **Usage:**

Called when resuming a game from a saved state.

### **Private Methods (Internal Workings)**

#### **FixedUpdate()**

Runs continuously in sync with Unity's physics updates.

- Increases the total elapsed time every frame.
- Updates the formatted game time string.
- Refreshes the `SaveCustomObject` with the latest data (time and scenes).

This ensures the save object always stays synchronized with the live game.

#### **LoadDataString(data)**

Receives the loaded data container and restores its values:

- Restores screenshot, game time, and elapsed time.
- Loads all saved items and scene names.
- Reloads scenes exactly as they were, in additive order (so multiple scenes reopen together).

#### **UpdateSaveCustomObject()**

Keeps the `SaveCustomObject` synchronized by:

- Updating its game time string.

- Rebuilding the list of all currently active scene names.

This ensures that the `SaveCustomObject` always reflects the current gameplay state.

### UpdateGameTime()

Converts the raw elapsed time (in seconds) into a readable clock format:

**hours:minutes:seconds.**

This makes the elapsed time presentable for UI or save menus.

### GetCamera()

Automatically finds a valid camera in the scene for capturing screenshots.

It checks in this order:

1. Player object (and its children).
2. Main camera in the scene.
3. Any camera found in the scene.

If no camera is found, it shows an error message in the Unity console.



### SaveCustomFile Structure

A simple data container used for serialization and deserialization.

It contains:

- **screenshot:** The captured image data.
- **gameTime:** The time the game had been running when the save occurred.
- **sceneNames:** All active scene names at the moment of saving.
- **saveCustomItems:** A list of custom items that were being saved.
- **elapsedTime:** The total gameplay time in seconds.

This structure ensures all key information about the session can be fully restored.



### Usage in a Game

1. **Attach the Script**  
Add it to a persistent `GameObject` (such as a save manager or system handler) in your scene.
2. **Assign References**  
Drag the **SaveCustomObject** (your save data handler) into the script's public field. Optionally, assign a camera if you don't want it to auto-detect.
3. **Automatic Tracking**  
The script begins counting gameplay time automatically.
4. **Saving**  
When a save event occurs (player saves or auto-save triggers), call the `SaveData()` method.
5. **Loading**  
When resuming the game, call the `LoadData()` method to restore the last saved state.
6. **Resetting**  
To clear or restart the save slot, call `ResetSave()`.



### In Summary

This class acts as a **scene-aware save manager**, combining:

- Time tracking,
  - Screenshot capture,
  - Scene recording,
  - JSON serialization,
  - File or `PlayerPrefs` storage,
- into a single automated system.

It's designed for flexibility: it can adapt to editor testing, standalone builds, or `PlayerPrefs`-only saves — all while ensuring the player's session can be accurately resumed later.

## Save Custom Object

### Overview

This script defines the **core data structure** of the “Save Custom Game” system. It stores all the **information related to game saves**, such as screenshots, game time, scene names, and any number of **custom variables** (vectors, floats, integers, strings, or booleans). It also includes **Unity Editor integration**, allowing developers to create a preconfigured asset directly from the menu, making it easy to manage and reuse across projects. The object created from this script is a **ScriptableObject**, which means it’s not tied to any specific scene and can be stored as a reusable asset in the project.

### Save Custom Object

This is the main container class that holds all the data related to the save system. It is designed to store:

- Screenshots of the game state.
- Scene information.
- Autosave settings.
- Saving location options.
- Lists of custom data (custom variables grouped by tags).

It’s the **central hub** that other parts of the system (like the scene manager or save manager) interact with.

### Screenshot and Scene Settings

These fields are grouped under the “Settings” header in the Unity Inspector.

- **screenshot**  
Stores an image of the game at the moment of saving.  
The image is converted into a byte array, allowing it to be serialized and restored later.
- **pixelLimit**  
Defines the maximum allowed size for the screenshot in pixels.  
This prevents memory overload by restricting image resolution.
- **gameTime**  
Keeps the formatted game timer string (for example, “01:12:09”).  
It represents how long the game session lasted at the moment of saving.
- **sceneNames**  
Contains a list of the names of all currently active or relevant scenes.  
This list is essential for correctly restoring multiple scenes during loading.

### Auto Save Settings

Grouped under “Auto Save Settings,” these variables define how automatic saving behaves.

- **autosaveEnabled**  
Toggles the entire autosave feature on or off.
- **saveGameByEvent**  
Enables autosaving when certain custom game events occur (for example, entering a checkpoint or finishing a level).
- **saveGameByTime**  
Enables autosaving after a fixed time interval.
- **saveInterval**  
Specifies how often the game should automatically save when the time-based autosave option is enabled (default is every 60 seconds).

These settings allow developers to build flexible save mechanisms that can work either automatically or by specific triggers.

### Saving Mode Settings

Grouped under “Saving Mode Settings,” these options determine **where** the data is stored.

- **gameData**  
When enabled, the game saves data in its own local storage structure (used for internal management or testing).
- **localLow**  
Saves data to the system's persistent path (for example, inside the LocalLow folder on Windows).  
This option ensures the save persists even if the game is reinstalled or updated.
- **playerPrefs**  
Uses Unity's built-in PlayerPrefs system to store save data as simple key-value pairs.  
This option is ideal for smaller projects or web builds.

Only one of these modes is typically active at a time, depending on how the developer configures the save system.

## Custom Items Settings

This section stores **lists of user-defined data** that the game wants to remember. Each entry can represent a set of values grouped under a shared tag, such as the player's progress, stats, or inventory.

- **saveCustomItems**  
A list containing multiple custom item groups.  
Each group can hold several different types of data: vectors, floats, integers, strings, and booleans.

This design makes the system modular — developers can store any kind of variable without having to modify the save system itself.

## Save Custom Item

Each custom item represents a **group of related values**, identified by a unique tag. It's like a container that groups together data belonging to a specific gameplay system or object.

- **itemTag**  
The identifier for the group (for example, "PlayerStats" or "LevelProgress").  
This tag allows the system to find and update this specific set of values later.
- **itemVector**  
Holds a list of all Vector4 values related to this tag.  
Useful for storing positions, rotations, or any 3D/4D data.
- **itemFloat**  
Holds a list of floating-point numbers, used for continuous values (like speed, health, or experience points).
- **itemInt**  
Holds a list of integers, used for whole numbers (like scores, item counts, or levels completed).
- **itemString**  
Holds a list of text-based values, such as player names, quest states, or identifiers.
- **itemBool**  
Holds a list of true/false values, used for flags (for example, "doorOpened" or "bossDefeated").

By using these lists, developers can save any kind of structured gameplay data, grouped by context.

## Value Containers

Each type of custom value (vector, float, int, string, or bool) is stored in its own dedicated structure.

Every one of them includes:

- A **tag**, to identify what the value represents.
- The **actual value**, which is stored and restored when loading.

### Vector Value

- **vectorTag** — The name used to identify this vector.
- **vectorValue** — The stored Vector4 data, which could represent position, rotation, or color.

#### Float Value

- **floatTag** — The name of the value.
- **floatValue** — The actual floating-point number saved.

#### Integer Value

- **intTag** — The label identifying the variable.
- **intValue** — The stored integer number.

#### String Value

- **stringTag** — The descriptive label.
- **stringValue** — The stored text value.

#### Boolean Value

- **boolTag** — The identifier for the boolean variable.
- **boolValue** — The true/false value stored.

This modular structure ensures that all custom data can be read, written, or expanded easily, without altering the main save system.

### Unity Editor Integration

The script also includes a **custom editor utility**, available only when running in the Unity Editor. This tool simplifies the creation of the save data asset.

#### Menu Functionality

A new option appears in the Unity Editor's "Assets" menu:

**Create → Save Custom Game → Save Custom Object Data**

When selected:

1. It ensures a **Resources** folder exists inside the project's Assets directory.
2. It checks whether an asset called "Save Custom Object Data" already exists.
  - If it does, a confirmation window appears asking if the user wants to replace it.
3. If approved (or if no previous asset exists), a new SaveCustomObject asset is created.
4. The new asset is automatically saved, refreshed, and highlighted in the Project view.

This allows developers to create or replace their main save configuration file instantly, without manually setting it up.

### Usage in a Game

#### 1. Asset Creation

- Open the Unity Editor.
- Use the "Assets → Create → Save Custom Game → Save Custom Object Data" menu to generate a SaveCustomObject asset.
- This asset becomes the main configuration file for the saving system.

#### 2. Linking to Scripts

- Other scripts (such as the scene save handler) reference this asset to read and write data.

#### 3. Custom Variables

- Developers can add as many custom items and variable types as needed to track player progress, preferences, or dynamic states.

#### 4. Runtime Behavior

- During gameplay, this object holds all live save data, which is then serialized to disk or PlayerPrefs when saving occurs.

### In Summary

The **Save Custom Object** script defines a **flexible, data-driven framework** for saving and loading any kind of game information.

It serves as:

- A **persistent data asset**, independent of scenes.

- A **customizable container**, capable of holding any mix of variables.
- A **bridge between gameplay and storage**, letting the save system handle data dynamically.
- A **Unity Editor-friendly structure**, ensuring that the setup process is straightforward and visual.

In short, it's the **foundation** of the entire "Save Custom Game" system — every save or load operation depends on this object as its central point of reference.

## Check Button Selected

This script is designed to ensure that the user interface (UI) always remains responsive, particularly when navigating with a keyboard or game controller. It does so by constantly monitoring the current selection in Unity's **Event System** and automatically re-selecting a predefined button if the current one becomes inactive or missing.

Let's break down how it works and what each part does:

### General Purpose

The script acts as a safeguard for UI navigation.

In Unity, UI elements like buttons can lose focus — for example, if a menu is opened or a button is deactivated. When this happens, the player might no longer be able to navigate the menu using the keyboard or controller.

This script ensures there's *a/ways* a selected button, so navigation never breaks.

### Variable Explanation

- **defaultButton**

This is a private reference to a **Button** component that the developer assigns in the Unity Inspector.

It represents the button that will automatically be selected when the UI has no currently active element.

It's marked with the attribute `SerializeField`, so it's visible and editable in the Unity Editor, even though it's private.

The `Tooltip` attribute helps developers understand its function when hovering over it in the Inspector.

### Property: DefaultButton

The property provides controlled access to the private `defaultButton` field:

- The **getter** allows other scripts to retrieve which button is currently set as the default.
- The **setter** allows other scripts to assign a new default button if needed.

This property is a clean and safe way to expose the variable for use by other components without making it fully public.

### Method: Update

This method runs once every frame during gameplay. It's responsible for checking and maintaining UI focus.

Here's how it works step by step:

1. **Checks if an Event System exists.**

Unity's **EventSystem** is what manages UI interactions.

If the scene doesn't contain one (for example, in a scene without any UI), the script stops immediately since there's nothing to check.

2. **Verifies the current selection.**

The script checks whether:

- The current selected object is null (meaning no UI element is selected), or
- The selected object exists but is **inactive** in the scene hierarchy.

Either case means the player has lost UI focus.

### 3. Re-selects the default button.

When no active selection exists, the script re-selects the predefined default button. This keeps the UI navigation active — especially important for players using controllers or keyboard navigation, where losing selection can break movement between buttons.

#### Usage in Practice

- Attach this script to a **GameObject** in your UI (it can be the **Canvas** or a manager object).
- Assign a button (for example, the “Start” button or “Resume” button) as the **defaultButton** in the Inspector.
- When the game runs, the script ensures that if no button is currently selected — or if the active one becomes disabled — it will automatically select this default button again.

#### Typical Scenarios

- **Main Menu Navigation:** When switching between menus, sometimes focus is lost — this script ensures a default option is always highlighted.
- **Controller Support:** Without it, players using a controller might lose the ability to navigate menus if the selected item disappears.
- **Pause Menu Systems:** When the pause menu opens or closes submenus, the **EventSystem** might lose focus — this script restores it automatically.

#### Summary

In essence, this script serves as a **UI failsafe**.

It keeps the user interface navigation functional by monitoring Unity’s **EventSystem** every frame and ensuring that a **default button** is always selected when needed.

The combination of serialized fields, properties, and per-frame checks makes it a lightweight but essential script for maintaining consistent and user-friendly menu navigation.

## Default Selected Button

This script is designed to maintain a consistent and intuitive **UI navigation experience** in Unity by ensuring that a specific button is automatically highlighted and selected whenever its associated **GameObject** becomes active. It is particularly useful in menus or UI panels where you want a default option to always be pre-selected when the user opens the interface.

Let’s go through how it works, the purpose of each element, and how it’s used:

#### Overall Function

When a UI panel or menu containing this script becomes active (for example, when a menu appears on screen), the script tells Unity’s **Event System** to focus on a predefined button.

This means that:

- The button will appear highlighted visually.
- It will be ready to receive input from a keyboard, controller, or navigation system immediately.

This eliminates cases where no button is selected, which can break controller or keyboard navigation.

#### Variable Explanation

##### button

- This is a **private reference** to a **Button** component.
- It’s marked with **SerializeField**, which means you can assign it directly in the Unity Inspector without making it publicly accessible.
- It’s also decorated with a **Tooltip**, which explains its purpose in the Inspector — it’s the button that will be automatically selected when the **GameObject** becomes active.

In simple terms, this variable stores *which button* should be highlighted and made active when the UI is enabled.

### Property: Button

This is a **public property** that allows other scripts to access or change the assigned button safely.

- The **getter** allows other parts of the program to check which button is currently set as the default.
- The **setter** allows other components to assign a new button dynamically at runtime.

This provides flexibility while maintaining encapsulation — you can modify which button is selected without exposing the internal variable directly.

### Method: OnEnable

This is a **Unity lifecycle method** that automatically runs whenever the GameObject the script is attached to becomes active in the scene.

That could happen when:

- A UI menu is opened.
- A scene loads and activates its UI elements.
- The object is re-enabled after being disabled.

Here's what happens inside it step by step:

1. **Clears the current UI selection.**

The script first resets Unity's **EventSystem** by removing any currently selected object. This prevents leftover selections from previous menus or scenes from interfering with the new one.

2. **Selects the assigned default button.**

After clearing, the script immediately tells Unity to select and highlight the button stored in the button variable.

This ensures that navigation (with keyboard or controller) begins from this specific button, giving the player a consistent starting point.

This method essentially *resets and reestablishes focus* every time the UI is shown.

### Usage in Practice

To use this script effectively:

1. **Attach it to the same GameObject as your UI menu or a relevant UI manager object.**
2. **Assign a Button** in the Inspector to the button field — typically the first button in your menu or the one you want to be pre-selected.
3. When the menu or UI panel becomes active in the game:
  - The EventSystem clears any previous selection.
  - The assigned button is automatically selected and highlighted.

### Typical Scenarios

- **Main Menu:** Automatically highlights the “Start Game” or “Continue” button when the menu opens.
- **Pause Menu:** Ensures “Resume” is always pre-selected when the pause screen appears.
- **Settings Screen:** Automatically focuses on the first option when the settings window becomes visible.
- **Controller Navigation:** Prevents navigation issues where no button is highlighted, causing inputs from a gamepad or keyboard to stop working.

### Summary

In short, this script ensures a smooth, professional user experience by:

- Automatically resetting Unity's UI selection system when a menu is activated.
- Highlighting a specific predefined button every time that menu becomes visible.
- Maintaining consistent UI focus for keyboard and controller users.

It acts as a **UI focus initializer**, guaranteeing that whenever a menu is shown, navigation always starts from a known and intended point.

## Test Save Custom

This script serves as a **runtime testing and debugging tool** for a custom save system within Unity.

Its purpose is to allow developers to **manually test saving, auto-save behavior, and object activation** while the game is running — without needing to trigger these actions through menus or UI.

It provides quick keyboard shortcuts for:

- Activating or disabling the **auto-save system**.
- Performing a **manual save**.
- **Toggling GameObjects** on and off to simulate in-game state changes.

All these operations print feedback in the Unity console, so the developer knows exactly what occurred.

### General Overview

When attached to any **GameObject** in the scene, this script continuously monitors the keyboard. When specific keys are pressed (defined in the Inspector), it performs certain test actions such as saving, enabling/disabling auto-save, or toggling **GameObjects**.

This is particularly useful during development to ensure the save system is working correctly without using the final UI implementation.

### Variable Breakdown

#### activateAndSave

- Represents the **keyboard key** used to both enable auto-save and trigger an immediate save.
- By default, this key is the **Spacebar**, but it can be changed from the Unity Inspector.
- When pressed, it tells the save system to start auto-saving and performs a manual save action to confirm it works.

#### disable

- Represents the **keyboard key** used to turn **off auto-save** during runtime.
- By default, it is the **Escape key**, but like the previous one, it can be adjusted in the Inspector.
- When this key is pressed, the script disables the auto-save system and prints a confirmation message in the Unity console.

#### gameObjects

- This is an **array of GameObjects** assigned in the Inspector.
- These objects are used for testing — the script can toggle their active state (on/off) dynamically.
- This simulates gameplay situations where objects appear or disappear, useful to test if the save system correctly tracks their state.

#### testSaveSlot

- An **optional integer variable** that defines which save slot to use when testing saves.
- If the value is above zero, the script prints a message showing which slot the save was triggered in.
- If no slot is specified (value  $\leq 0$ ), it simply performs a generic save.

### Method Breakdown

#### Update

- This method runs once per frame automatically.
- It calls the internal function `HandleInput`, which checks whether any of the defined keys have been pressed during that frame.
- It acts as the continuous listener that waits for user input to trigger test actions.

### HandleInput

- This method handles all **keyboard input detection** for the script.
- It checks:
  - If the **activation key** (for example, Spacebar) is pressed → enables auto-save and performs a test save.
  - If the **disable key** (for example, Escape) is pressed → disables auto-save and logs this in the console.
- The two main actions here simulate enabling/disabling save functionality during gameplay, allowing developers to confirm that auto-save is being controlled correctly.

### SaveTestSlot

- This method performs the **actual saving action**.
- It first checks whether a test save slot number is set (greater than zero).
  - If it is, it calls a **SaveEvent** (a function provided by the save system utility) and prints a message indicating which slot was used.
  - If no slot number is set, it performs a save without referencing any slot, simply confirming the save happened.
- This provides visual confirmation through the Unity console that a save event was successfully triggered.

### SwitchGameObject

- This is a **public method**, meaning it can be called from other scripts or Unity UI buttons.
- When executed, it **toggles the active state** of every `GameObject` in the `gameObjects` array:
  - If an object is active → it becomes inactive.
  - If it's inactive → it becomes active again.
- After finishing the toggle, it prints a message to the console confirming the change.
- This allows developers to test how the save system reacts to changing object states in real time (for example, whether it correctly saves which objects are active).

### Usage Summary

1. **Attach the script** to any `GameObject` in your scene — typically one used for debugging or testing.
2. In the Unity **Inspector**, assign:
  - The keys for **activating/saving** and **disabling** auto-save.
  - Any **GameObjects** you want to toggle.
  - (Optionally) a **save slot number** to test with.
3. **Play the scene**:
  - Press the assigned **activation key** → enables auto-save and triggers a save event.
  - Press the **disable key** → disables auto-save.
  - Optionally call `SwitchGameObject()` (for example, through a button or editor test) → toggles visibility/activation of the chosen objects.
4. Watch the **Unity Console** for feedback messages that confirm each action worked as expected.

### Practical Use Cases

- **Debugging save logic:** Ensures that auto-save and manual save events are triggered correctly.

- **Testing scene data persistence:** Confirms that object states are saved and restored properly.
- **Controller or UI testing:** Can be tied to in-game buttons to simulate save behavior.
- **Development quality control:** Helps verify that save events are correctly connected to the SaveDataUtility system.

### In Short

This script acts as a **developer testing tool** that provides hands-on control over the save system while the game is running.

It allows toggling auto-save, triggering manual saves, and testing object states with simple keyboard shortcuts — all while giving instant feedback in the console.

It's not meant for final gameplay use, but rather as an efficient way for developers to **validate save and auto-save functionality during development**.

## Paged Menu Base TMP and Paged Menu Base

### Overview — What This Script Is

This file defines an **abstract base class** used to create save/load menus that can display **multiple pages** of save slots (for example page 1, page 2, page 3...).

It prepares everything the interface needs:

- Page navigation
- Slot display
- Screenshot preview rendering
- Label formatting
- Confirmation boxes
- Input field validation
- Cleanup of textures
- Utility helpers for derived classes

The class **does not perform the actual save or load action**.

Instead, it requires child classes to define what happens when a slot is clicked or confirmed.

### STRUCT: SlotUI

A group of UI components representing a single save slot.

Each slot contains:

1. **button**
  - The clickable button that triggers the save/load action.
2. **image**
  - A preview screenshot shown inside the slot.
3. **label**
  - The text containing the slot name or metadata.
4. **path**
  - The file path for this slot if using file-based saving.

This struct is repeated for each slot in the menu.

### Inspector Fields (Configured in Unity)

These are elements you assign from the Unity Editor.

#### ✓ Confirmation Panel Elements

- **confirmationPanel:** The window that appears when confirming overwrite or load.
- **confirmButton:** The button that approves the action.
- **cancelButton:** The button that closes the confirmation panel.

#### ✓ Save Limit System

- **maxSavePages:** Maximum allowed number of pages (–1 means unlimited).

#### ✓ Title System

- **titleLoad**: The text showing which mode or page is active.
- **text**: Label for normal pages (e.g., "Page").
- **textAutomatic**: Label for the autosave page (page zero).

#### ✓ Slots

- **slots**: An array holding 6 slot UI groups.

#### ✓ Page Navigation

- **right**: Button to advance to the next page.
- **left**: Button to return to the previous page.
- **inputField**: Where the player types a page number manually.



#### Internal Variables

- **saveCustomInScene**: Reference to the SaveCustom component in this scene.
- **currentNumber**: The currently visible page number.
- **initializedUI**: Prevents UI from being wired twice.
- **prefsKeyBase**: Key used to remember the last page with PlayerPrefs.



#### Public Properties

These simply expose inspector variables so other scripts can read or change them. They handle:

- Panel
- Buttons
- Page settings
- Title text
- Slot array
- Navigation inputs

Useful if external systems need to modify the menu at runtime.



#### Unity Lifecycle Methods

##### ✓ OnEnable

Runs when the menu opens. It does three things:

1. Finds the SaveCustom component in this scene.
2. Restores the last viewed page if saved previously.
3. Sets up the UI elements and loads all slot data.

##### ✓ OnDisable

Runs when the menu closes.

- Clears textures inside all slot images to prevent memory leaks.



#### UI Setup Methods

##### ✓ SetupUI

Runs only once.

It:

1. Connects the right and left page buttons to navigation functions.
2. Connects input field to number validation and end-edit logic.
3. Connects cancel button to close the confirmation panel.
4. Adds click listeners to every slot so they call a generic handler.

Prevents duplicate setup by checking initializedUI.

##### ✓ SetupInitialPage

When the menu opens:

- Sets the input field text equal to the current page
- Calls slot refresh



#### Page Title Handling

## ✓ UpdateTitle

Changes the title text depending on the page:

- Page 0 → shows the autosave label
- Others → show normal page label

## 📄 Page Navigation Methods

### ✓ IncreasePage

Advances the page number:

- Checks if not above limit
- Updates page number
- Refreshes slots

### ✓ DecreasePage

Goes backward one page:

- Stops at page zero
- Refreshes slots

### ✓ ValidateNumericInput

Allows only digit characters to be typed into the page input field.

### ✓ OnEndEditInputField

When the player types a page number manually:

- Converts the text into a valid number
- Clamps it within the limits
- Refreshes slots

## 🔧 Slot Refresh System

### ✓ RefreshSlots

The core UI refresh routine:

1. Validates page number limits
2. Saves the current page to PlayerPrefs
3. Enables or disables left/right navigation buttons
4. Updates slot labels ("page - slotNumber")
5. Clears previous textures
6. Calls **LoadSlotsData**, which child classes must implement
7. Updates the page title

This ensures the slot visuals always match the selected page.

### 📌 Abstract Method: LoadSlotsData

Must be implemented in child classes.

Its job is:

- Load save data for the current page
- Determine whether each slot should show a screenshot, metadata, or be empty
- Call ApplySlotData where necessary

This method changes depending on whether the child is a Save menu or Load menu.

## 📁 Slot Input Methods

### ✓ OnSlotPressed

Runs when a slot button is clicked:

1. Ensures the slot is valid and interactable
2. Calls the child class's action handler

### ✓ OnCancelPressed

Closes the confirmation panel.

### ✓ HandleSlotAction (abstract)

Child class decides what happens when a slot is clicked.

Examples:

- In a save menu → open confirmation if overwriting
- In a load menu → immediately load or request confirmation

### ✓ **ConfirmAction (abstract)**

Runs when the user presses the confirm button.

### □ **Confirmation Helper**

### ✓ **ShowConfirmationForSlot**

Used by child classes to show the confirmation panel:

1. Activates the confirmation window
2. Removes old listeners from confirm button
3. Adds a listener calling ConfirmAction with the correct slot

### 🎨 **Slot Helper Methods**

### ✓ **SetSlotActive**

Enables or disables a slot visually and interactively.

- When inactive → slot becomes transparent and not clickable
- When active → slot is visible and clickable

### ✓ **GetSavePath**

Creates the full file path for a given save name.

Takes into account:

- JSON extension
- Whether the saving system uses PlayerPrefs or files
- Target location (persistent data folder, editor folder, etc.)

### ✓ **ApplySlotData**

Writes loaded save data into the slot UI:

- Updates label with metadata (including game time)
- Converts screenshot bytes into a texture
- Enables the slot

Used by child classes during LoadSlotsData.

### 🔧 **Cleanup**

### ✓ **ClearSlotTextures**

Destroys all textures assigned to slot images.

Prevents memory from accumulating when switching pages or menus.

### 📋 **Usage Summary**

You use this base class like this:

1. Create a child class (for example SaveMenu or LoadMenu).
2. Implement these two required methods:
  - LoadSlotsData
  - HandleSlotAction
  - ConfirmAction
3. Assign all the UI elements in the Unity Inspector.
4. When the player opens the menu:
  - It loads the page
  - Updates interface
  - Shows screenshot previews
5. Player can:
  - Navigate pages (left/right/input)
  - Click a slot
  - Confirm or cancel actions

This base class handles **everything visual and functional**, letting child classes define only the actual save/load logic.

## Load Menu Manager TMP and Load Menu Manager

### Explanation of How the Load Menu Script Works

This script represents a **load menu** for a save system.

Its goal is to display pages of save slots, show screenshots and metadata, and allow the player to confirm and load a save file.

It works by **extending** the base class PagedMenuBase, which already contains all the shared logic such as:

- Managing pages
- Refreshing UI
- Rendering slot images
- Handling numeric input
- Clearing textures
- Navigating pages

This script only provides the parts **specific to loading**:

how to read save data from disk or PlayerPrefs, and what happens when the user selects a slot.

### How the Script Works (Step by Step)

#### 1. When the Menu Opens

When the menu is enabled, the script simply calls the base class initialization.

The base class does:

- Restores the last page number
- Sets up all UI events (buttons, input field, slot buttons)
- Updates the initial page
- Loads the initial slot data
- Updates the page title

This class doesn't add extra behavior during startup.

#### 2. Loading Slot Information for the Current Page

The core behavior of this class is inside the method that loads slot data for each page.

#### How slots are processed

For each slot (1 to 6):

1. The script builds a file name depending on:
  - The page number
  - The slot index

For example:

"3 - 2" → page 3, slot 2.

2. It uses the base class helper to generate the full save path for that file.
3. It stores this path in the slot's path field.
4. Then it checks which save system is being used:
  - **PlayerPrefs mode** (single string per slot stored in memory)
  - **File mode** (save files stored on disk)

#### If PlayerPrefs is used

- If there is no entry for this slot, the slot is disabled.
- If the entry exists:
  - The script extracts the stored JSON string
  - Converts the JSON into a save-file structure
  - Uses the inherited method to:
    - Update the label
    - Render the screenshot

- Mark the slot as active

### If File Save Mode is used

- If the expected file does not exist:
  - The slot is disabled.
- If the file exists:
  - The script reads the JSON text from disk
  - Converts it into a save-file structure
  - Uses the inherited method to apply the data into the UI

### 3. What Happens When a Slot Is Pressed

When the user clicks on a slot:

1. The script checks:
  - The slot must have an image
  - The slot must have a screenshot (texture assigned)

If either is missing, this means the slot is empty → nothing happens.

2. If the slot is valid, the menu opens a **confirmation panel**.

The base class provides a helper that:

- Shows the confirmation panel
- Replaces the confirm button's function so it loads the chosen slot

This ensures that players cannot accidentally load a game without confirming.

### 4. What Happens When the Player Confirms

When the confirm button is pressed:

1. The script tells the save system:
  - Which file name to load
  - What path should be used (empty if PlayerPrefs mode)
2. It commands the central save-loading component to perform the loading.
3. Finally, it hides the confirmation panel again.

From here, the gameplay scene or game state is restored depending on how the save system works internally.

### Variables and What They Represent

This class inherits all variables from the base class, like:

- Page number
- Slot structures
- UI elements (buttons, input field, images, labels)
- Confirmation panel
- Page navigation
- Save system references

This class **adds no new variables** of its own.

It only uses the inherited ones.

Important inherited variables it uses directly:

#### slots

A list of slot elements.

Each entry contains:

- A button
- An image
- A label
- A path (filled by this script)

#### inputField

Where the user can manually type the page number.

## saveCustomInScene

The component responsible for loading and saving data.  
Used here to trigger the load action.

## confirmationPanel / confirmButton

Used when the user clicks a slot that contains data.

## Usage Summary

### What this component does

- Shows pages of save slots
- Reads save data (from memory or files)
- Displays screenshots and info
- Lets the user select a slot
- Asks for confirmation
- Loads the chosen save when confirmed

### How a player interacts with it

1. Opens the load menu
2. Moves through pages using left/right or typing a page number
3. Clicks on a slot
4. Confirms the choice
5. The game loads that save

### How a developer uses it

- Add this component to a UI GameObject
- Ensure the base class UI fields are filled:
  - Slots (buttons, labels, images)
  - Confirmation panel
  - Page navigation buttons
  - Input field
- Provide a SaveCustomInScene component in the same scene
- Ensure save files or PlayerPrefs entries are created by the save system

## Save Menu Manager TMP and Save Menu Manager



### Overview

This script represents a **Save Menu Manager**, built on top of a base class called *PagedMenuBase*.

Its job is to:

- Show a paginated list of save slots.
- Display thumbnails or empty slots depending on whether data exists.
- Allow saving into a selected slot.
- Show a confirmation dialog if the target slot is already used.
- Support two saving methods:
  - Using system files.
  - Using PlayerPrefs.

It is marked as a **legacy** version that uses the *Paged Menu* architecture.



### Class-Level Components

The class inherits everything from *PagedMenuBase*, which already provides:

- A list of UI slots.
- A current page number.
- Input field for naming save files.
- Confirmation panel.

- Logic related to navigation between pages.

This script only provides **saving-specific behavior**.

## **Variables inherited and referenced**

Although the script does not declare new variables, it uses these inherited ones:

### **slots**

- A list of UI slot elements.
- Each slot contains:
  - A button
  - An image (thumbnail)
  - A path string

### **inputField**

- Optional text field used for naming save files.
- If empty, the system uses the current page number.

### **currentNumber**

- Represents the active page or index used when naming files.

### **saveCustomInScene**

- Object responsible for performing the actual save operation.
- Contains:
  - A data model
  - A boolean deciding between PlayerPrefs or files
  - A method that saves the data
  - A field storing the final filename

### **confirmationPanel**

- A UI panel that appears when the system asks the player if they want to overwrite an existing save.

## **Unity Life Cycle**

### **OnEnable**

- Runs when the object becomes active in the scene.
- Simply ensures that the base class performs its setup first.
- Nothing custom happens here.

## **Slot Loading System**

### **LoadSlotsData**

This method runs when a page is displayed or refreshed.

Its purpose is to:

1. Inspect each slot of the current page.
2. Determine whether data exists for that slot.
3. If data exists:
  - Load and apply it visually.
  - Show the thumbnail or icon.
4. If no data exists:
  - Mark the slot as empty and transparent.

### **How it works (step by step)**

For each slot:

1. The script generates a file name in the format:
2. [InputFieldText or PageNumber] - [SlotNumber]
3. The script gets a full path for the save file.
4. The script assigns this path to the slot.

Then depending on the save method:

### **Case 1 — PlayerPrefs**

- If PlayerPrefs does not contain an entry with the file name:
  - The slot is treated as empty.

- If PlayerPrefs contains data:
  - The JSON is loaded and converted into a save structure.
  - The slot visual is updated using this data.

## Case 2 — Files

- If the file does not exist:
  - Slot is treated as empty.
- If it exists:
  - The JSON file is read from disk.
  - Converted into a save structure.
  - Slot visuals are updated.

## Slot Interaction

### HandleSlotAction (when a slot is pressed)

This handles the player tapping or clicking on a slot.

1. The system checks if the slot already has a thumbnail.
  - If the slot has a texture, it means it's **occupied**.
  - If not, it's **empty**.
2. If the slot is **occupied**:
  - The confirmation panel appears.
  - The player must confirm overwriting.
3. If the slot is **empty**:
  - The save operation occurs immediately.

## Visual Rules for Save Mode

### SetSlotActive

This method is a bit different from a load menu because:

- In save mode, **all slots must stay interactable**.
- The button is always enabled.

What gets visually changed:

- If the slot has a texture:
  - Image is fully visible.
- If not:
  - Image becomes fully transparent, not invisible (so layout stays intact).

This makes empty slots look like empty placeholders instead of disabled buttons.

## Saving Process

### PerformSave

This method performs the actual save operation:

1. Builds a final file name in the same format used when loading.
2. Tells the *saveCustomInScene* system to save the data using that name.
3. Refreshes the UI:
  - Clears all thumbnails.
  - Reloads slot data to show fresh results.

There is no confirmation here; this method only executes.

## Confirming an Overwrite

### ConfirmAction

This runs only when the player presses the confirmation button.

1. Calls the same save process as a normal save.
2. Hides the confirmation panel afterward.

## Usage Summary

To use this system in a Unity UI:

1. Add the component to a menu screen.

2. Provide:
  - Slot UI elements
  - A confirmation panel
  - A save system reference
  - Optional input field for naming saves
3. Open the menu using the pagination base system.
4. When the user selects a slot:
  - If empty → save immediately.
  - If occupied → ask confirmation.
5. After saving:
  - The slot thumbnail updates automatically.